# LAB EXERCISES & DEEP DIVES

## Hands-On Practice with 2026 Features

Automatic Clustering · Type Widening · Serverless Jobs

LAB A

# LAB A: Automatic Liquid Clustering Practice

## Prerequisites

- Unity Catalog-enabled workspace
- Databricks Runtime 15.4 LTS or higher
- CREATE SCHEMA and CREATE TABLE permissions

## Exercise 1: Create Table with CLUSTER BY AUTO

**■ Exercise 1: Create Auto-Clustered Table**

```
# Step 1: Setup
USE CATALOG main;
USE SCHEMA default;

# Step 2: Create sample sales table
CREATE TABLE IF NOT EXISTS sales_auto (
  order_id BIGINT,
  customer_id BIGINT,
  product_id BIGINT,
  amount DOUBLE,
  order_date DATE,
  region STRING
) USING DELTA CLUSTER BY AUTO
AS SELECT
  ROW_NUMBER() OVER (ORDER BY RAND()) AS order_id,
  (RAND() * 1000)::BIGINT AS customer_id,
  (RAND() * 100)::BIGINT AS product_id,
  RAND() * 500 AS amount,
  DATE_ADD(CURRENT_DATE(), CAST((RAND() * -365) AS INT)) AS order_date,
  CASE WHEN RAND() < 0.25 THEN 'US-East'
       WHEN RAND() < 0.5 THEN 'US-West'
       WHEN RAND() < 0.75 THEN 'EMEA'
       ELSE 'APAC' END AS region
FROM (SELECT 1 FROM RANGE(100000)) t;

# Step 3: Verify clustering is enabled
DESCRIBE DETAIL sales_auto;
-- Look for: clusteringColumns = [order_id, customer_id] (auto-selected)
```

## Exercise 2: Compare Manual vs Auto Clustering

**■ Exercise 2: Query Comparison**

```
# Create manual cluster table (for comparison)
CREATE TABLE sales_manual (
  order_id BIGINT, customer_id BIGINT, product_id BIGINT,
  amount DOUBLE, order_date DATE, region STRING
) USING DELTA CLUSTER BY (customer_id, order_date)
AS SELECT * FROM sales_auto;

# Run queries to observe optimization benefits
```

```
EXPLAIN FORMATTED
SELECT * FROM sales_auto
  WHERE customer_id = 42 AND order_date >= "2025-01-01";

# Compare with manual clustering
EXPLAIN FORMATTED
SELECT * FROM sales_manual
  WHERE customer_id = 42 AND order_date >= "2025-01-01";

# Both should show data skipping; AUTO may use different columns
```

**Observation:** Run OPTIMIZE on both tables and compare file statistics. AUTO clustering may choose different columns based on your actual query patterns.

LAB B

# LAB B: Auto Loader Type Widening & cleanSource

## Exercise 3: Type Widening in Schema Evolution

■ **Exercise 3: Type Widening**

```
# Setup: Create volumes for source and checkpoint
CREATE VOLUME IF NOT EXISTS main.default.al_source;
CREATE VOLUME IF NOT EXISTS main.default.al_checkpoint;

# Create initial Parquet file with INT amount column
from pyspark.sql import Row
initial_data = spark.createDataFrame([
  Row(order_id=1, amount=100),  # amount is INT
  Row(order_id=2, amount=250)
])
initial_data.write.parquet(
  "/Volumes/main/default/al_source/initial/")

# Step 2: Auto Loader with Type Widening (Runtime 16.4+)
(spark.readStream.format("cloudFiles")
  .option("cloudFiles.format", "parquet")
  .option("cloudFiles.schemaLocation",
          "/Volumes/main/default/al_checkpoint/schema")
  # NEW: Enable type widening
  .option("cloudFiles.schemaEvolutionMode",
          "addNewColumnsWithTypeWidening")
  .load("/Volumes/main/default/al_source")
  .writeStream
  .option("checkpointLocation",
          "/Volumes/main/default/al_checkpoint/ckpt")
  .mode("append")
  .toTable("orders_silver"))

# Step 3: Add new file with BIGINT amount
new_data = spark.createDataFrame([
  Row(order_id=3, amount=9999999999)  # BIGINT
])
new_data.write.parquet(
  "/Volumes/main/default/al_source/new_batch/")

# Check schema: amount should be BIGINT (widened automatically)
spark.sql("DESCRIBE orders_silver").show()
```

## Exercise 4: cleanSource for Automatic Cleanup

■ **Exercise 4: cleanSource Cleanup**

```
# Auto Loader with automatic file cleanup
(spark.readStream.format("cloudFiles")
  .option("cloudFiles.format", "json")
  .option("cloudFiles.schemaLocation",
          "/Volumes/main/default/al_checkpoint/schema_json")
  # NEW: Delete files after successful ingestion
  .option("cloudFiles.cleanSource", "delete")
  .load("/Volumes/main/default/al_source/json_landing")
```

```
    .writeStream
    .option("checkpointLocation",
            "/Volumes/main/default/al_checkpoint/json_ckpt")
    .trigger(availableNow=True)
    .toTable("events_silver"))

# Alternative: Move to archive instead of delete
# .option("cloudFiles.cleanSource", "move")
# .option("cloudFiles.cleanSourceArchiveDir",
#         "/Volumes/main/default/archive")

# Verify files have been deleted or moved
dbutils.fs.ls("/Volumes/main/default/al_source/json_landing")
# Should be empty after successful run
```

**Caution:** Using cleanSource="delete" is irreversible. Always test with "move" first to archive, then verify data integrity before switching to "delete" in production.

LAB C

# LAB C: Creating Serverless Jobs (Lakeflow)

## Exercise 5: Serverless Job Submission

■ **Exercise 5: Serverless Job Submit**

```
# Create a simple transformation notebook and submit as Serverless Job

# Notebook: /Users/me@company.com/transform_orders
# ---
import pyspark.sql.functions as F

orders_df = spark.read.table("main.raw.orders")
clean_orders = (orders_df
  .filter(F.col("order_id").isNotNull())
  .withColumn("order_year", F.year(F.col("order_date")))
  .select("order_id", "customer_id", "amount", "order_year"))
clean_orders.write.mode("overwrite").option("overwriteSchema","true")
  .saveAsTable("main.silver.orders")
print("■ Orders loaded to silver")

# Submit via Python SDK (auto-uses Serverless)
from databricks.sdk import WorkspaceClient

w = WorkspaceClient()
job = w.jobs.create(
  name="Load Orders - Serverless",
  tasks=[{
    "notebook_task": {
      "notebook_path": "/Users/me@company.com/transform_orders"
    },
    "task_key": "load_orders"
  }]
)
print(f"Job ID: {job.job_id}")

# Run job (defaults to Serverless in UC workspace)
run = w.jobs.run_now(job_id=job.job_id)
print(f"Run started: {run.run_id}")
```

## Comparison: Serverless vs Classic Job Cluster

| Attribute | Serverless Job | Classic Job Cluster | All-Purpose Cluster |
|---|---|---|---|
| Startup time | < 3 seconds | 2–5 minutes | 1–2 minutes (if running) |
| Cost model | Per-second execution | DBU/hour (incl. idle) | DBU/hour (must stay running) |
| Permission needed | None (UC user) | CREATE CLUSTER | CREATE CLUSTER or inherited |
| Auto-termination | Immediate after job | After job completes | Manual or timeout |
| Default (2026 UC) | ■ YES | — | — |
| Photon enabled | ■ Always | Optional | Optional |
| Scaling | Automatic | Manual (config) | Manual (instance pool) or auto |

**Key Insight:** Serverless is the 2026 default for Lakeflow Jobs in UC workspaces. Use classic only if you need custom Spark config or libraries not pre-installed in Serverless.

# Deep Dive: Feature Tradeoffs & Use Cases

## When to Use CLUSTER BY AUTO vs Manual

| Scenario | Use AUTO? | Reason |
|---|---|---|
| Query patterns unknown | ■ YES | AUTO adapts to actual queries |
| High-volume, variable access | ■ YES | AUTO self-tunes without manual ALTER |
| Stable, high-cardinality column (e.g., customer_id) | ■ NO | Manual wins with known patterns |
| Table < 100TB | ■ YES | AUTO sufficient; no ingestion-time clustering needed |
| Frequent schema/workload changes | ■ YES | AUTO evolves; manual requires rewrites |

## Type Widening Use Cases

| Source Change | Type Widening Solves | Example |
|---|---|---|
| Integer overflow | int → long without rewrite | order_qty: 100 → 5,000,000 |
| Decimal precision | decimal(10,2) → decimal(20,4) | price: $99.99 → $9,999.99 |
| Floating-point loss | float → double | metric: 1.5 → 1.123456789 |
| No rewrite penalty | Schema evolution without data copy | Multiple widening steps allowed |

## cleanSource Compliance Matrix

| Requirement | delete | move | default (none) |
|---|---|---|---|
| Fast storage cleanup | ■ | ■ | ■ |
| Audit trail preserved | ■ | ■ | ■ |
| Recovery possible | ■ | ■ (from archive) | ■ |
| Compliance (GDPR/HIPAA) | ■ (if delete required) | ■ (audit + delete later) | Risk |
| Cost optimization | ■ (immediate) | ~ | ■ (storage stays) |

# Exam Tips: Updated for 2026

**Tip 1:** CLUSTER BY AUTO is now a default recommendation

If the exam says "optimize a table without manual config," think AUTO, not Z-ORDER.

**Tip 2:** Type Widening prevents schema evolution failures

A question about int→long changes with Auto Loader? Answer: addNewColumnsWithTypeWidening

**Tip 3:** cleanSource is new; test your Qs carefully

Distinguish "delete" (irrev.) vs "move" (audit). Compliance scenarios prefer "move".

**Tip 4:** Serverless Jobs are 2026 default

Any Lakeflow Job question in UC workspace → assume Serverless unless otherwise stated.

**Tip 5:** Automatic features = Databricks-managed

AUTO and Serverless shift ops burden from you to Databricks. Exam often tests this shift.